

05 · Walk-forward y sobreajuste

Quant-Backtester · material de estudio

Hasta ahora has medido estrategias con parámetros "de libro" (50/200, 20 días y 2σ), decididos sin mirar los datos. En cuanto pruebas combinaciones y te quedas con la mejor, el backtest deja de medir solo la estrategia: mide también tu búsqueda. En este capítulo verás por qué el mejor resultado de una búsqueda está sesgado hacia arriba aunque ninguna combinación tenga ventaja real, y construirás la herramienta estándar para obtener una cifra honesta: la validación *walk-forward*.

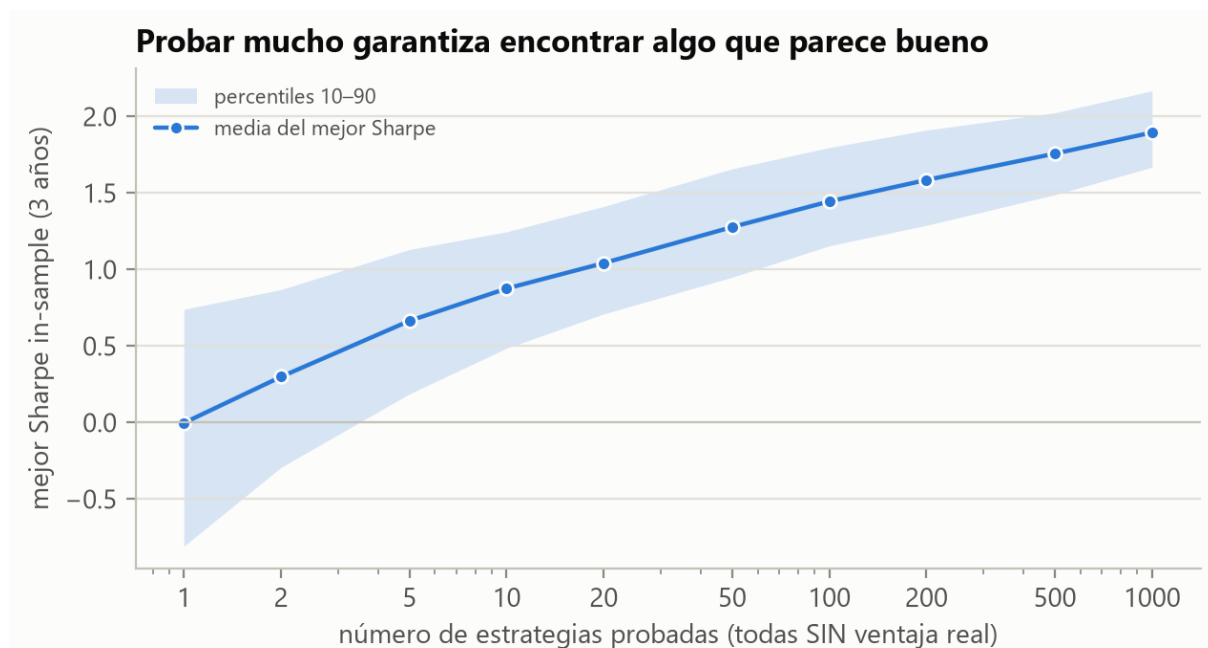
El capítulo cubre los cuatro TODO de `src/walkforward.py`: **5.1** `generate_windows` (las ventanas de entrenamiento y test), **5.2** `evaluate_params` (puntuar una combinación en un tramo sin mirar el futuro y con las medias ya "calientes"), **5.3** `grid_search` (evaluar y ordenar una rejilla) y **5.4** `walk_forward` (encadenar los tramos de test en una única curva fuera de muestra). Necesitas tener hechos los TODO de los capítulos 1 a 4: el walk-forward usa el motor, las estrategias y las métricas.

1. Intuición

In-sample y out-of-sample. Un dato es *in-sample* (IS) para una decisión si ha intervenido en ella, y *out-of-sample* (OOS) si no. Si eliges `fast = 20`, `slow = 150` porque es la combinación con mejor Sharpe en 2010-2020, ese Sharpe ya no mide lo que la estrategia hará: mide lo bien que esa combinación se ajusta a esos datos, ruido incluido. Es como corregir un examen cuyas preguntas el alumno conocía de antemano: la nota mide memoria, no comprensión.

Cada backtest es una medición ruidosa. El Sharpe que calculas con unos años de datos es el Sharpe verdadero más un error de estimación. Con rendimientos diarios aproximadamente independientes, ese error tiene una desviación típica de alrededor de $1/\sqrt{T}$, con T en años: ± 1 con un año, $\pm 0,58$ con tres, $\pm 0,32$ con diez. Un Sharpe verdadero de 0,5, que sería una estrategia razonable, queda enterrado bajo el ruido de una ventana de tres años.

Data snooping y sesgo de selección. Ahora prueba N combinaciones y quédate con la mejor. Aunque ninguna tenga ventaja real (Sharpe verdadero 0 en todas), la ganadora no tendrá Sharpe 0: tendrá el máximo de N errores de estimación, y ese máximo crece con N. Elegir el máximo es elegir la combinación a la que más ha favorecido el ruido. Esto se llama *data snooping* (fisgar en los datos) o sesgo de selección, y no requiere mala fe: basta con probar muchas cosas e informar solo de la que mejor sale.



Mejor Sharpe in-sample (3 años de datos) entre N estrategias aleatorias sin ventaja real, en función del número de estrategias probadas N ; la banda va del percentil 10 al 90. Aunque el Sharpe verdadero de todas es 0, el del ganador crece con N .

Las cifras están en el apartado 2. El resumen: con 17 combinaciones independientes y 3 años de entrenamiento, el ganador tiene de media un Sharpe in-sample cercano a 1 sin ninguna ventaja; con 1000 pruebas, cercano a 1,9. Un mejor Sharpe in-sample de 1,1 en tu rejilla no es, por sí solo, evidencia de nada.

La idea del Deflated Sharpe Ratio. Bailey y López de Prado (2014) formalizaron esta corrección en el *Deflated Sharpe Ratio* (DSR). La idea cabe en una frase: en lugar de preguntar si el Sharpe observado es mayor que 0, pregunta si es mayor que el Sharpe máximo que esperarías obtener *por azar* dado el número de pruebas que has hecho, la longitud de la muestra y la forma de la distribución de los rendimientos (la asimetría y las colas gruesas hacen el Sharpe más ruidoso). El resultado es una probabilidad: la de que el Sharpe verdadero sea positivo una vez descontada la búsqueda. Dos consecuencias prácticas. Primera: tienes que **contar todas las pruebas**, también las que descartaste; cuantas más hagas, más alto debe estar el listón. Segunda: lo que cuenta es el número *efectivo* de pruebas; 20/150 y 20/200 comparten casi todas sus operaciones, así que valen bastante menos que dos pruebas independientes.

Por qué los mapas de calor son "estrechos e inestables". En el notebook 03 pintarás el Sharpe en train de cada combinación (fast, slow) en varias ventanas. Verás dos cosas. *Estrechos*: la casilla ganadora suele ser un pico con vecinos claramente peores. Si la ventaja fuera real, pasar de `slow = 150` a `slow = 200` no debería hundirla: verías una meseta, no un pico. *Inestables*: el pico cambia de sitio de una ventana a otra, aunque dos ventanas de train consecutivas comparten dos de sus tres años. La explicación es la de antes: las diferencias verdaderas entre combinaciones vecinas son de décimas de Sharpe como mucho y el ruido de estimación es de $\pm 0,58$. El máximo de una superficie ruidosa cae donde el ruido ha sido más generoso, y el ruido cambia con los datos.

La respuesta: walk-forward. Si vas a elegir, al menos mide la elección con datos que no ha visto. El walk-forward lo hace de forma sistemática: elige los parámetros con los 3 años anteriores, aplícalos el año siguiente, avanza un año y repite. Cada día de la curva

resultante se operó con parámetros elegidos solo con información disponible en ese momento. Fíjate en lo que mide: no "el Sharpe de 20/150", sino el de un **procedimiento** ("cada año, usa la mejor combinación de los últimos 3 años"), que es justo lo que habrías podido ejecutar en tiempo real.

Las tres curvas del notebook 03. Sobre el mismo periodo fuera de muestra (desde el primer año de test hasta el final de los datos) compararás:

- **Parámetros fijos (50/200).** Decididos sin mirar los datos. Honesta, aunque arbitraria.
- **Optimizado in-sample.** La mejor combinación de `grid_search` sobre *todo* el histórico, aplicada al periodo OOS. Es trampa: para elegirla se usaron los mismos años en los que se evalúa. Casi seguro que sale la mejor de las tres, y esa ventaja no era alcanzable.
- **Walk-forward.** La curva de este capítulo.

Lo habitual es que la in-sample quede arriba y que el walk-forward quede bastante por debajo, a menudo al nivel de los parámetros fijos o peor (además paga los costes de cambiar de parámetros). La distancia entre la curva in-sample y la walk-forward estima cuánto del "rendimiento" era solo selección.

El walk-forward también se puede sobreajustar. El walk-forward protege la elección de *parámetros*, pero hay decisiones por encima: la rejilla, la longitud de las ventanas, la métrica que se maximiza, la familia de estrategias, los activos. Si cambias cualquiera de ellas mirando la curva walk-forward hasta que queda bonita, conviertes el periodo OOS en in-sample de una búsqueda de nivel superior y vuelves al punto de partida con otro nombre. Las defensas son de disciplina, no de código: fija esas decisiones antes de mirar el resultado, apunta cuántas variantes pruebas (es la N del DSR) y, si puedes, reserva un último tramo de datos que no toques hasta el final.

2. Formalización

Ventanas. Sea t_0 la primera fecha del índice (`index[0]`) y t_N la última (`index[-1]`). Llama Y_{tr} , Y_{te} e Y_s a `train_years`, `test_years` y `step_years`. En las fórmulas, sumar años a una fecha significa sumar años **de calendario** (`pd.DateOffset(years=...)`), no 365 días ni 252 sesiones. La ventana $k = 0, 1, 2, \dots$ empieza en

$$a_k = t_0 + k \cdot Y_s$$

y sus dos tramos son intervalos semiabiertos (el extremo derecho no se incluye):

$$\begin{aligned} \text{train}_k &= [a_k, a_k + Y_{tr}) \\ \text{test}_k &= [a_k + Y_{tr}, a_k + Y_{tr} + Y_{te}) \end{aligned}$$

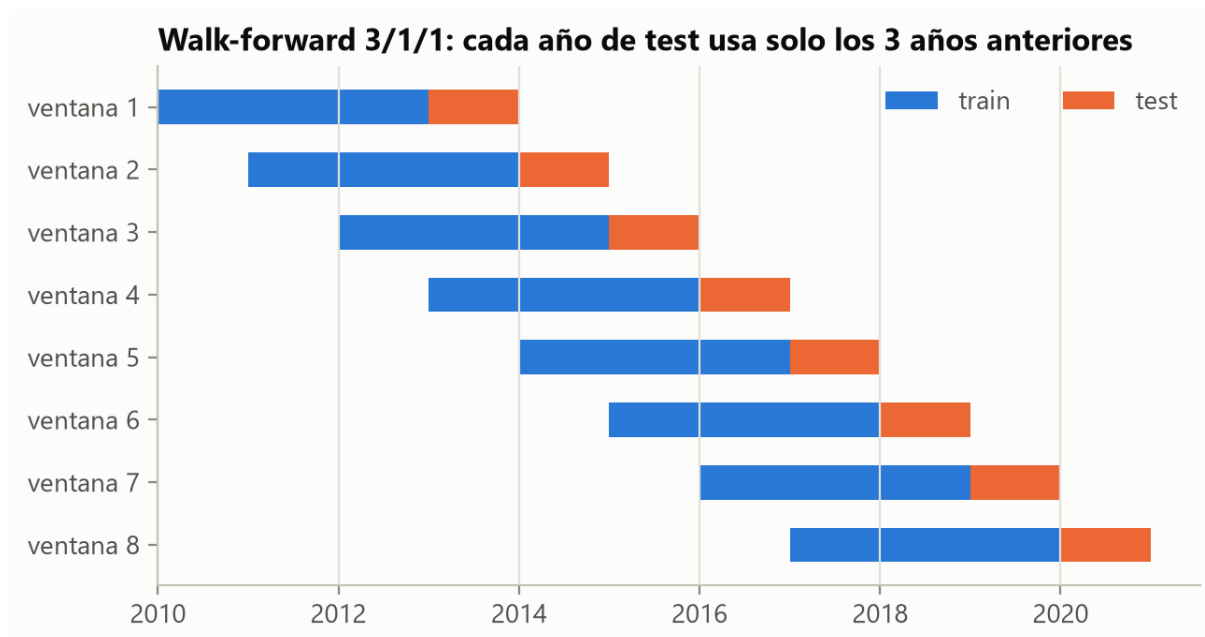
Se generan ventanas mientras el test empiece dentro de los datos:

$$a_k + Y_{tr} \leq t_N$$

La última ventana puede tener el test incompleto (su fin cae después de t_N): se usa lo que haya.

Por qué semiabiertos. El fin del train es exactamente el inicio del test. Con [inicio, fin), ninguna fecha cae en los dos tramos; con intervalos cerrados, el día frontera serviría para elegir los parámetros y para evaluarlos. **Por qué $Y_s \geq Y_{te}$.** Si el paso fuera menor que el test, los tramos de test se solaparían y algunos días contarían dos veces en la curva encadenada; `generate_windows` ya lo rechaza con `ValueError`. Con $Y_s = Y_{te}$ los tramos de

test se tocan sin solaparse; con Y_s mayor que Y_{te} quedan huecos entre ellos.



Ventanas walk-forward con 3 años de train, 1 de test y paso de 1 año. Cada fila es una ventana; los tramos de test, uno detrás de otro, forman la curva fuera de muestra.

Ejemplo a mano. Con $t_0 = 2010-01-04$ (el primer día hábil de 2010, donde empiezan los datos reales), $t_N = 2015-06-30$ y 3/1/1:

k	train	test	¿inicio del test $\leq t_N$?
0	[2010-01-04, 2013-01-04)	[2013-01-04, 2014-01-04)	sí
1	[2011-01-04, 2014-01-04)	[2014-01-04, 2015-01-04)	sí
2	[2012-01-04, 2015-01-04)	[2015-01-04, 2016-01-04)	sí (incompleto)
3	[2013-01-04, 2016-01-04)	[2016-01-04, 2017-01-04)	no: se para

Salen tres ventanas. Observa que los "años" no van de enero a diciembre, sino de 4 de enero a 4 de enero: están anclados en la primera fecha.

Anclar en t_0 , no encadenar. Cada a_k se calcula desde t_0 , no sumando Y_s al inicio anterior. La diferencia aparece con el 29 de febrero: sumar un año a 2012-02-29 da 2013-02-28 (no existe el 29), y si a partir de ahí sigues sumando, el 29 se ha perdido para siempre.

k	anclado: $t_0 + k$ años	encadenado: anterior + 1 año
0	2012-02-29	2012-02-29
1	2013-02-28	2013-02-28
2	2014-02-28	2014-02-28
3	2015-02-28	2015-02-28
4	2016-02-29	2016-02-28

Tampoco sirve `Timedelta(days=365)`: tres "años" de 365 días desde 2010-01-04 acaban el 2013-01-03, porque 2012 tuvo 366 días. Y contar 252 filas mezcla calendario con sesiones: no todos los años tienen 252.

Puntuación de una combinación. Sea θ una combinación de parámetros (un dict de la rejilla), M una métrica de `METRICS` y $r_t(\theta)$ el rendimiento neto del día t en un backtest de la

estrategia con parámetros θ . La puntuación en el tramo $[u, v)$ es

$$S(\theta; u, v) = M(r_t(\theta) : u \leq t < v)$$

con una condición crucial sobre cómo se obtiene $r_t(\theta)$: la estrategia recibe **todos** los precios con fecha anterior a v , desde el principio de los datos, y después M se aplica **solo** a los días del tramo. Si recortaras los precios a $[u, v)$ antes de calcular las señales, una media de 200 sesiones no tendría valor hasta la sesión 200 del tramo: en un año de test de unas 252 sesiones, alrededor del 79 % de los días sin posición. Operando de verdad eso no pasaría, porque la media ya estaría calculada con los meses anteriores. Además, la estrategia empezaría el tramo fuera de mercado y pagaría dentro de él una entrada desde 0 que nunca ocurrió: la posición ya venía de antes. Los datos anteriores a u son pasado: usarlos no es mirar el futuro. Mirar el futuro sería que la estrategia recibiera precios del día v o posteriores.

Selección. En cada ventana se elige la combinación que maximiza la puntuación en train:

$$\theta_k^* = \operatorname{argmax}_{\theta} S(\theta; a_k, a_k + Y_{tr})$$

Con empates gana la primera de la rejilla, y una puntuación NaN (por ejemplo, el Sharpe de una estrategia que nunca entra, cuya σ es 0) nunca gana.

Encadenado. La posición walk-forward de un día t del tramo de test k es la posición que habría tenido la estrategia con los parámetros elegidos en esa ventana:

$$w_t^{WF} = w_t(\theta_k^*)$$

y 0 fuera de los tramos de test. Con esas posiciones se hace **un único** backtest. En el primer día t de un tramo, el turnover compara la posición nueva con la del día anterior, que venía de la ventana previa:

$$\tau_t = |w_t(\theta_k^*) - w_{t-1}(\theta_{k-1}^*)|$$

Ejemplo: si la ventana $k-1$ eligió estar largo (+1) y la k elige estar corto (-1), $\tau = 2$ y, con 5 + 5 pb, el coste de ese día es $2 \times 10/10\,000 = 0,002$, un 0,2 % en ese activo. Si en vez de encadenar posiciones hicieras un backtest independiente por tramo y pegaras los rendimientos, en el backtest del tramo k el día anterior también se calcularía con θ_{k-1}^* , la posición previa ya sería -1 y ese 0,2 % desaparecería. Con cambios de parámetros frecuentes, esos costes ocultos se acumulan. El primer día fuera de muestra se entra desde 0, así que también paga: $\tau = |w|$.

README frente a código

El README habla de "concatenar los tramos de test". El docstring de `walkforward.py` precisa qué se concatena: posiciones, no rendimientos. Manda el código, por el motivo que acabas de ver.

El tamaño del sesgo de selección. Si el Sharpe verdadero es 0, el Sharpe anualizado estimado con T años de datos diarios es aproximadamente normal, con desviación típica

$$\sigma(\hat{S}) \approx \frac{1}{\sqrt{T}}$$

y el máximo de N estimaciones independientes tiene esperanza

$$E\left(\max_{i \leq N} \hat{S}_i\right) \approx \frac{z_N}{\sqrt{T}}$$

donde z_N es la esperanza del máximo de N normales estándar. Crece muy despacio, pero sin límite; una cota sencilla es

$$z_N \leq \sqrt{2 \ln N}$$

Esperanza del mejor Sharpe in-sample cuando ninguna estrategia tiene ventaja (z_N calculado exactamente):

N pruebas	z_N	T = 1 año	T = 3 años	T = 10 años
1	0,00	0,00	0,00	0,00
10	1,54	1,54	0,89	0,49
17	1,79	1,79	1,04	0,57
100	2,51	2,51	1,45	0,79
1000	3,24	3,24	1,87	1,03

Dos lecturas: el sesgo crece con N (despacio, como $\sqrt{\ln N}$) y decrece con la longitud de la muestra (como $1/\sqrt{T}$). La fila de 17 es tu rejilla del cruce de medias con ventanas de 3 años. Como las combinaciones están muy correlacionadas, el N efectivo es menor que 17 y el sesgo algo menor, pero el orden de magnitud, cerca de 1, es el que debes tener en la cabeza al leer la columna `train_sharpe` de `selections`.

3. Del papel al código

Lo que ya está hecho

`src/walkforward.py` trae resueltas las estructuras de datos y dos utilidades. Conócelas antes de empezar, porque tus funciones las crean o las consumen.

- `Window`: dataclass inmutable (`frozen=True`) con cuatro `pd.Timestamp`: `train_start`, `train_end`, `test_start` y `test_end`, con los fines exclusivos. `train_mask(index)` y `test_mask(index)` devuelven un array booleano de numpy con `True` en las fechas de `index` que caen en [inicio, fin); sirve para seleccionar filas con `.loc`. `str(w)` imprime `train [2010-01-04, 2013-01-04) test [2013-01-04, 2014-01-04)`. Dos ventanas con las mismas fechas son iguales con `==`, que es lo que comparan los tests.
- `GridSearchResult`: `table` (una fila por combinación, ordenada de mejor a peor), `best_params` (el dict ganador, tal cual estaba en la rejilla), `best_score` (su puntuación) y `metric` (el nombre de la métrica).
- `WalkForwardResult`: `result` (el `BacktestResult` del periodo fuera de muestra), `windows` y `searches` (una búsqueda por ventana, en el mismo orden). Su propiedad `selections` monta una tabla con una fila por ventana: `train_start`, `test_start`, `test_end`, los parámetros elegidos y la puntuación en train en una columna que se llama `train_` seguido del nombre de la métrica, por ejemplo `train_sharpe`.
- `expand_grid(param_grid, constraint=None)`: producto cartesiano de un dict de listas, devuelto como lista de dicts en el orden de `itertools.product` (el último parámetro varía más deprisa). `constraint` descarta combinaciones. Con `config.PARAM_GRIDS["sma_crossover"]` salen $4 \times 5 = 20$ combinaciones; la restricción `fast < slow` elimina (50, 50), (100, 50) y (100, 100), y quedan las 17 del notebook. Los valores conservan su tipo: `fast` sigue siendo `int`.
- `heatmap_table(search, index, columns)`: convierte la tabla de una búsqueda con dos parámetros en una matriz (un parámetro en filas, otro en columnas, la puntuación en las casillas), lista para el mapa de calor. Las combinaciones que no existen, como las descartadas por la restricción, quedan como `NaN`.

Ejemplo con la rejilla `expand_grid({"fast": [10, 20], "slow": [100, 200]})`, cuyas posiciones son 0 = (10, 100), 1 = (10, 200), 2 = (20, 100) y 3 = (20, 200), y unas puntuaciones inventadas de 0,42, NaN, 0,87 y 0,42:

search.table				heatmap_table(search, "fast", "slow")		
	fast	slow	score	slow	100	200
2	20	100	0.87	fast		
0	10	100	0.42	10	0.42	NaN
3	20	200	0.42	20	0.87	0.42
1	10	200	NaN			


```
search.best_params -> {"fast": 20, "slow": 100}
search.best_score  -> 0.87
```

El índice de la tabla (2, 0, 3, 1) es la posición de cada combinación en la rejilla original. El empate entre las posiciones 0 y 3 se resuelve a favor de la 0, que aparece antes, y el NaN va al final.

TODO 5.1 · generate_windows

Qué hace. Recibe el índice de fechas y devuelve la lista de `Window` del walk-forward, según las fórmulas del apartado 2. La validación de los argumentos y la conversión a `DatetimeIndex` ya están escritas antes del TODO.

Firma.

```
def generate_windows(
    index: pd.DatetimeIndex,
    train_years: int = config.WF_TRAIN_YEARS,
    test_years: int = config.WF_TEST_YEARS,
    step_years: int = config.WF_STEP_YEARS,
) -> list[Window]:
```

Convenciones que comprueban los tests.

- Con días hábiles de 2010-01-01 a 2020-12-31 y 3/1/1 salen 8 ventanas. La primera tiene train [2010-01-01, 2013-01-01) y test [2013-01-01, 2014-01-01); la última, test [2020-01-01, 2021-01-01) (`test_generate_windows_count_and_first_last`).
- En cada ventana `train_end == test_start` y, con paso igual al test, el `test_end` de una ventana es el `test_start` de la siguiente (`test_generate_windows_contiguous_and_disjoint`).
- Con paso de 2 años, los tests empiezan en 2013, 2015, 2017 y 2019 (`test_generate_windows_step_two_years`).
- Si los datos acaban el 2020-06-30, la última ventana tiene `test_start` 2020-01-01: un test incompleto también cuenta (`test_generate_windows_includes_partial_last_window`).
- Si no cabe ninguna ventana (datos de 2010-01-01 a 2012-06-30), lista vacía (`test_generate_windows_too_short_is_empty`).
- Con datos desde el 2012-02-29 y 1/1/1, la ventana 4 empieza el 2016-02-29: los inicios están anclados en `t0` (`test_generate_windows_anchored_offsets`).
- Paso menor que el test, `ValueError` (`test_generate_windows_rejects_overlapping_tests`). Este test ya pasa antes de que escribas nada, porque la validación está hecha.

Pseudocódigo.

1. (Hecho) Valida los años y convierte index en `DatetimeIndex`.
2. `t0` = primera fecha del índice; `t_last` = última fecha.
3. `k` = 0; ventanas = lista vacía.
4. Repite:
 - 4.1 `inicio` = `t0 + (k · step_years)` años de calendario <- siempre desde `t0`
 - 4.2 `fin_train` = `inicio + train_years` años
 - 4.3 `fin_test` = `fin_train + test_years` años
 - 4.4 si `fin_train` es posterior a `t_last`: sal del bucle
 - 4.5 añade `Window(inicio, fin_train, fin_train, fin_test)`
 - 4.6 `k` = `k + 1`
5. Devuelve ventanas (vacía si ni la primera cabe).

Pistas.

Pista 1 · Qué define una ventana

Todo se deriva del inicio de la ventana. La condición de parada se mira sobre el inicio del test (que es el fin del train), no sobre el fin del test: si exigieras que el test cupiera entero, perderías la última ventana incompleta.

Pista 2 · Herramientas

`pd.DateOffset(years=n)` suma años de calendario a un `Timestamp`, y con `n = 0` deja la fecha igual. `index[0]` e `index[-1]` ya son `Timestamp`. Como no sabes de antemano cuántas ventanas caben, un bucle `while` es lo natural. `Window(...)` recibe las cuatro fechas en el orden `train_start, train_end, test_start, test_end`.

Pista 3 · El detalle fino

El inicio de la ventana `k` es `t0` más un único `DateOffset` de `k * step_years` años; nunca el inicio de la ventana anterior más un `DateOffset` de `step_years`. La parada es con "posterior estricto": un test que empieza exactamente en la última fecha del índice sí se genera.

TODO 5.2 · evaluate_params

Qué hace. Devuelve un número: la puntuación de una combinación de parámetros en el tramo `[start, end)` según la métrica elegida. Es la pieza que usa `grid_search` y, a través de ella, `walk_forward`. La comprobación de la métrica y la conversión de `start` y `end` a `Timestamp` ya están hechas.

Firma.

```
def evaluate_params(
    prices: pd.DataFrame,
    strategy: Callable[..., pd.DataFrame],
    params: dict,
    start,
    end,
    metric: str = config.WF_METRIC,
    commission_bps: float = config.COMMISSION_BPS,
    slippage_bps: float = config.SLIPPAGE_BPS,
    lag: int = config.SIGNAL_LAG,
) -> float:
```

Convenciones que comprueban los tests.

- La puntuación coincide, con tolerancia relativa de `1e-10`, con la del backtest de **todo** el histórico recortado a `[START, END)`: las medias tienen que llegar calientes al tramo (`test_evaluate_params_uses_history_for_warmup_and_excludes_end`). Funciona porque una estrategia causal da las mismas señales con todos los datos que con los anteriores a `end`.
- `END` es el 2014-01-02, un día hábil, a propósito: si lo incluyes, la puntuación cambia y el mismo test lo detecta.
- La estrategia nunca recibe precios con fecha igual o posterior a `end` (`test_evaluate_params_never_sees_the_future`, que espía la fecha máxima que le llega a la estrategia).
- Se usa la métrica pedida: con `"sharpe"` y con `"cagr"` salen valores distintos (`test_evaluate_params_metric_choice`).
- Métrica desconocida, `KeyError` (hecho).

Pseudocódigo.

1. (Hecho) Comprueba que `metric` existe; `start` y `end` pasan a `Timestamp`.
2. `historia` = las filas de `prices` con fecha estrictamente anterior a `end`.
3. `señales` = la estrategia aplicada a `historia`, con `params` como argumentos con nombre.
4. `resultado` = `backtest` completo de `historia` con esas señales (mismos costes y lag).
5. `tramo` = rendimientos netos del resultado con fecha `t` tal que `start <= t < end`.
6. Devuelve la métrica `METRICS[metric]` aplicada a `tramo`.

Pistas.

Pista 1 · Dos preguntas distintas

Separa "¿qué datos ve la estrategia?" (todo lo anterior a `end`) de "¿qué días se puntúan?" (solo `[start, end]`). El primer recorte evita mirar el futuro; el segundo decide qué se mide. No recortes nada por `start` antes de calcular las señales.

Pista 2 · Herramientas

Una máscara booleana sobre el índice de fechas, usada con `.loc`, para los dos recortes. `strategy(datos, **params)` desempaqueta el dict como argumentos con nombre. `run_backtest` (capítulo 2) ya aplica el desfase y cobra los costes; los rendimientos netos están en su atributo `.returns`. `METRICS[metric]` es una función que recibe esa Series y devuelve un float.

Pista 3 · El detalle fino

`.loc[:end]` incluye `end` (con etiquetas entran los dos extremos), así que no sirve. Para la `historia`, compara el índice con `end` usando "menor estricto"; para el `tramo`, "mayor o igual que `start`" y "menor estricto que `end`" a la vez. No trates aparte el primer día del tramo: como la `historia` previa está incluida, su `turnover` es el real y no una entrada ficticia desde 0.

TODO 5.3 · grid_search

Qué hace. Puntúa cada combinación de la rejilla con `evaluate_params` en `[start, end]`, las ordena de mejor a peor y devuelve un `GridSearchResult`. La conversión de `grid` a lista y el error con la rejilla vacía ya están hechos.

Firma.

```
def grid_search(
    prices: pd.DataFrame,
    strategy: Callable[..., pd.DataFrame],
    grid: Sequence[dict],
    start,
    end,
    metric: str = config.WF_METRIC,
    commission_bps: float = config.COMMISSION_BPS,
    slippage_bps: float = config.SLIPPAGE_BPS,
    lag: int = config.SIGNAL_LAG,
) -> GridSearchResult:
```

Convenciones que comprueban los tests.

- Mayor puntuación, mejor; los NaN, al final. Con la rejilla `[{"level": 0}, {"level": -1}, {"level": 1}]` en un tramo alcista, el índice de la tabla es `[2, 1, 0]`: largo, corto y, al final, "siempre fuera", cuyo Sharpe es NaN (`test_grid_search_orders_best_first_nan_last`).
- `best_score` es la puntuación de la primera fila y `metric` guarda el nombre de la métrica (mismo test).

- Las columnas son exactamente los parámetros, en el orden de los dicts, y después `"score"`; una fila por combinación (`test_grid_search_table_columns`).
- El índice es la posición en la rejilla original y cada `score` coincide con `evaluate_params` de `grid[posición]` (`test_grid_search_scores_match_evaluate_params`).
- Con empates gana la que aparece antes en la rejilla, y la tabla conserva ese orden (`test_grid_search_ties_keep_grid_order`).
- `best_params` es el dict original de la rejilla, con sus tipos: `window` sigue siendo `int` (`test_grid_search_returns_original_dicts_with_original_types`).

Pseudocódigo.

1. (Hecho) `grid` pasa a lista; rejilla vacía -> `ValueError`.
2. Para cada combinación de `grid`, en orden, calcula su puntuación con `evaluate_params` en `[start, end)`, pasando `metric`, los costes y el lag. Guarda las puntuaciones.
3. Ordena las POSICIONES 0, 1, ..., n-1 (no las puntuaciones) de forma estable: primero las de puntuación válida, de mayor a menor; después las NaN.
4. Tabla: una fila por combinación con sus parámetros y la columna `"score"`, con índice = posición en la rejilla; reordena sus filas según el paso 3.
5. mejor = primera posición del orden.
6. Devuelve `GridSearchResult`(tabla, `grid[mejor]`, puntuación de mejor, `metric`).

Pistas.

Pista 1 · Ordena posiciones

Si ordenas posiciones en vez de puntuaciones, cada posición te da a la vez la etiqueta de la fila en la tabla y el dict original (`grid[posición]`). Así nunca necesitas reconstruir un dict a partir de la tabla.

Pista 2 · Herramientas

`sorted()` de Python es estable y acepta `key=`; `math.isnan` o `np.isnan` detectan NaN. `pd.DataFrame(grid)` crea una columna por clave, en el orden de los dicts, a la que añades `"score"`. Para reordenar filas por etiquetas del índice, `.loc[orden]`. Si prefieres hacerlo en pandas, `sort_values("score", ascending=False, kind="stable")` también vale y deja los NaN al final por defecto; sin `kind="stable"` no tienes garantizado el orden de los empates.

Pista 3 · El detalle fino

Cualquier comparación con NaN da `False`, así que una clave con NaN desordena la lista sin avisar: `sorted([0.87, nan, 0.42])` devuelve `[0.87, nan, 0.42]`, tal cual. Usa como clave una tupla cuyo primer elemento diga si la puntuación es NaN (Python compara tuplas elemento a elemento y `False < True`) y cuyo segundo elemento ordene de mayor a menor sin recurrir a `reverse` (piensa en el signo). Y `best_params` sale de `grid`, nunca de una fila de la tabla: una fila con columnas `int` y una columna `float` se convierte entera a `float`, y `rolling(20.0)` falla.

TODO 5.4 · walk_forward

Qué hace. El walk-forward completo: en cada ventana, búsqueda en train, señales con los parámetros ganadores y posiciones del tramo de test; después, un único backtest con todas las posiciones encadenadas y el recorte al periodo fuera de muestra. Ya están hechas la generación de ventanas (con error si no cabe ninguna) y el dict `costs` con comisión y slippage.

Firma.

```
def walk_forward(
    prices: pd.DataFrame,
    strategy: Callable[..., pd.DataFrame],
    grid: Sequence[dict],
    train_years: int = config.WF_TRAIN_YEARS,
    test_years: int = config.WF_TEST_YEARS,
    step_years: int = config.WF_STEP_YEARS,
    metric: str = config.WF_METRIC,
    commission_bps: float = config.COMMISSION_BPS,
    slippage_bps: float = config.SLIPPAGE_BPS,
    lag: int = config.SIGNAL_LAG,
) -> WalkForwardResult:
```

Convenciones que comprueban los tests. Casi todos usan `constant_level`, una estrategia de juguete de `tests/helpers.py` que da siempre la misma señal `level`, con la rejilla `[{"level": 1}, {"level": -1}]` sobre precios sintéticos de 2010-2020 con un régimen alcista o bajista por año.

- 8 ventanas y 8 búsquedas; los rendimientos tienen exactamente las fechas desde el 2013-01-01 hasta el final, sin duplicados (`test_walk_forward_structure`).
- La búsqueda de cada ventana coincide con `grid_search` en `[train_start, train_end)` (`test_walk_forward_selects_with_train_data`).
- En cada tramo de test, la posición es el `level` elegido en esa ventana (`test_walk_forward_applies_selected_params_in_each_test`).
- El turnover del primer día fuera de muestra es 1, porque se entra desde 0, y el del primer día de cada tramo es `|level nuevo – level anterior|`: el cambio de parámetros paga costes (`test_walk_forward_charges_costs_when_params_change`).
- Si se alteran los precios desde el 2017-01-01, no cambian las búsquedas de las ventanas cuyo train acaba en esa fecha o antes, ni los rendimientos anteriores a 2017 (`test_walk_forward_selection_is_not_affected_by_future_data`).
- Con `sma_crossover` de verdad, `selections` tiene `fast`, `slow` y `train_sharpe`, y la equity no tiene NaN (`test_walk_forward_with_real_strategy_runs`).

Pseudocódigo.

1. (Hecho) `windows = generate_windows(...)`; error si está vacía; `costs = {comisión, slippage}`.
2. Para cada ventana `w`, en orden:
 - 2.1 búsqueda = `grid_search` en `[w.train_start, w.train_end)` con `metric`, `costes` y `lag`.
 - 2.2 Guarda la búsqueda.
 - 2.3 historia = precios con fecha anterior a `w.test_end`.
 - 2.4 señales = estrategia(historia, con los mejores parámetros de la búsqueda).
 - 2.5 posiciones = `signals_to_positions`(señales, `lag`).
 - 2.6 Guarda solo las filas de posiciones con fecha en `[w.test_start, w.test_end)`.
3. Junta los tramos guardados, uno detrás de otro (no se solapan: `step >= test`).
4. Lleva ese bloque al índice completo de `prices`, con 0 en las fechas sin tramo de test.
5. `completo = backtest_positions`(`prices`, esas posiciones, `comisión`, `slippage`).
6. `fuera = completo` recortado desde la primera hasta la última fecha de los tramos de test.
7. Devuelve `WalkForwardResult(result=fuera, windows=windows, searches=búsquedas)`.

Pistas.

Pista 1 · Qué se encadena

Se encadenan posiciones, y el backtest se hace una sola vez, al final, sobre el índice completo. Así el motor ve el día frontera como un día cualquiera en el que la posición cambia y cobra su turnover. En ningún momento calculas los rendimientos de un tramo por separado.

Pista 2 · Herramientas

`signals_to_positions` (TODO 2.1) para el desfase, sin hacer el shift a mano.
`window.test_mask(df.index)` con `.loc` para quedarte con las filas del tramo. `pd.concat` para unir una lista de DataFrames. `reindex(..., fill_value=0.0)` para llevarlos al índice de `prices`.
`backtest_positions` acepta `**costs`. `BacktestResult.slice(inicio, fin)` recorta y rehace la equity desde 1.

Pista 3 · El detalle fino

`slice` funciona como `.loc`: incluye los dos extremos. Pásale la primera y la última fecha del bloque de posiciones concatenadas, no el `test_end` de la última ventana, que es exclusivo. Si seleccionas los tramos con `.loc[test_start:test_end]` en vez de con la máscara, el día frontera entra en dos tramos, aparecen fechas duplicadas y `reindex` falla. Con `step_years` mayor que `test_years` quedan huecos entre tramos: el relleno con 0 los deja fuera de mercado y siguen dentro del periodo recortado.

4. Errores típicos

Recortar los precios al tramo antes de calcular las señales

Si en `evaluate_params` cortas los precios a `[start, end)` y después llamas a la estrategia, las medias pierden su historia: con `slow = 200` casi todo el año queda sin posición y se paga una entrada ficticia desde 0. Síntoma: la puntuación no coincide con la del backtest completo recortado y las combinaciones lentas salen sistemáticamente peor. Lo detecta `test_evaluate_params_uses_history_for_warmup_and_excludes_end`.

Incluir el día end

`prices.loc[:end]` o `returns.loc[start:end]` incluyen `end`. Síntoma: la estrategia recibe un día del futuro y la métrica puntúa un día de más. Lo detectan `test_evaluate_params_never_sees_the_future` y el test de calentamiento (por eso `END` es un día hábil).

Encadenar DateOffset o usar Timedelta

Sumar un año al inicio anterior pierde el 29 de febrero para siempre; `Timedelta(days=365)` se desplaza un día con cada bisiesto. Síntoma: inicios de ventana uno o varios días antes de lo esperado. Lo detectan `test_generate_windows_anchored_offsets` y `test_generate_windows_count_and_first_last`.

Exigir que el test quepa entero

Si paras cuando el fin del test supera la última fecha, pierdes la ventana incompleta del final. Síntoma: 7 ventanas en lugar de 8 con datos hasta el 2020-12-31, porque el test de 2020 acaba el 2021-01-01. Lo detectan `test_generate_windows_count_and_first_last` y `test_generate_windows_includes_partial_last_window`.

Ordenar con NaN en la clave o de forma inestable

Una clave con NaN deja la lista a medio ordenar; un algoritmo no estable puede alterar el orden de los empates. Síntoma: un NaN en medio de la tabla, o gana otra combinación empatada. Lo detectan `test_grid_search_orders_best_first_nan_last` y `test_grid_search_ties_keep_grid_order`. Ojo: este último puede pasar por casualidad con un orden inestable, así que no te fíes de él.

Perder el índice original de la tabla

Si construyes la tabla ya ordenada con un índice nuevo 0, 1, 2... o haces `reset_index`, el índice deja de ser la posición en la rejilla. Síntoma: índice `[0, 1, 2]` donde se esperaba `[2, 1, 0]`, y puntuaciones que no casan con `grid[posición]`. Lo detectan `test_grid_search_orders_best_first_nan_last` y `test_grid_search_scores_match_evaluate_params`.

Reconstruir best_params desde la tabla

`table.iloc[0]` mezcla parámetros enteros con un `score` float, así que la fila entera se vuelve float y `window` pasa a valer 20.0. Síntoma: en el walk-forward, `rolling()` falla porque la ventana no es entera. Lo detecta `test_grid_search_returns_original_dicts_with_original_types`.

Pegar rendimientos en vez de posiciones

Un backtest por tramo seguido de `pd.concat` de los rendimientos olvida el coste de pasar de los parámetros de una ventana a los de la siguiente. Síntoma: turnover 0 en días frontera en los que la posición pasa de +1 a -1. Lo detecta `test_walk_forward_charges_costs_when_params_change`.

Dejar NaN fuera de los tramos de test

Si no rellenas con 0 las fechas que no están en ningún tramo (el primer periodo de train), `backtest_positions` lanza `ValueError` con el mensaje "positions contiene NaN o infinitos". Todos los tests de `walk_forward` caen con ese error.

Devolver el backtest sin recortar

Si devuelves el resultado sobre el índice completo, la curva incluye los años de train con posición 0 y las métricas se diluyen. Lo detecta `test_walk_forward_structure`, que exige que los rendimientos empiecen el primer día de test.

5. Preguntas y ejercicios

1. ¿Por qué una fecha no puede estar a la vez en el train y en el test de una ventana? ¿Qué sesgo introduciría el día frontera si estuviera en los dos?
2. Con `step_years = 2` y `test_years = 1`, ¿qué ocurre en los huecos entre tramos de test? ¿Cómo afectan esos días al CAGR y al Sharpe de la curva walk-forward?
3. Con `slow = 200` y un tramo de 252 sesiones recortado antes de calcular la señal, ¿cuántos días habría posición como mucho? Recuerda que la posición llega un día después de la señal.
4. En la primera ventana, el train empieza en la primera fecha de los datos y no hay historia previa para el calentamiento. ¿Qué combinaciones se ven afectadas en esa ventana y en qué sentido?

5. Has probado 17 combinaciones con 3 años de train. Según la tabla del apartado 2, ¿qué Sharpe in-sample esperarías del ganador si ninguna tuviera ventaja? Compáralo con la columna `train_sharpe` de `wf_sma.selections`.
6. ¿Por qué decimos que el walk-forward mide un procedimiento y no un parámetro? A la vista de `plot_param_stability`, ¿tiene sentido hablar de "el parámetro óptimo"?
7. Con 5 + 5 pb, ¿cuánto cuesta en un activo cada cambio de parámetros que lleva la posición de +1 a -1? ¿Y si la posición pasa de 1 a 0?
8. **Ejercicio (notebook 03, celda de la curva tramposa).** Haz el `grid_search` sobre todo el histórico (el fin es exclusivo: usa la última fecha más un día), backtestea con los mejores parámetros y recorta con `.slice(oos_start, None)`. Compara las tres curvas y calcula cuánto Sharpe desaparece al pasar de in-sample a walk-forward.
9. **Ejercicio (notebook 03).** Para cada ventana de `wf_sma`, puntúa *todas* las combinaciones también en su tramo de test con `evaluate_params` y calcula la correlación de rangos entre el orden en train y el orden en test (`corr(method="spearman")`). Si elegir en train sirviera de algo, ¿qué signo y qué tamaño esperarías? ¿Qué obtienes?
10. **Ejercicio (notebook 03).** Repite el walk-forward con `train_years=1`, con `train_years=8` y con `metric="calmar"`. Apunta *antes* de ejecutar qué esperas que pase. Después, pregúntate si elegirías la configuración que mejor sale, y relee el último párrafo del apartado 1.

6. Checkpoint

```
python -m pytest tests/test_walkforward.py      # los 21 tests de este capítulo
python -m pytest -k generate_windows            # solo TODO 5.1 (7 tests)
python -m pytest -k evaluate_params            # TODO 5.2 (3 tests + 1 de grid_search
    cuyo nombre lo contiene)
python -m pytest -k grid_search                 # solo TODO 5.3 (5 tests)
python -m pytest -k walk_forward               # solo TODO 5.4 (6 tests)
python -m pytest -k walk_forward -rs          # además, el motivo de cada skipped
python -m pytest -m extension                  # solo la extensión de volatility
    targeting (capítulo 06)
python -m pytest -m "not extension"            # todo el proyecto salvo la extensión
```

Un test que llega a un TODO sin implementar no sale como fallo, sino como *skipped* con el mensaje "TODO pendiente: TODO 5.x · ...", y al final de la salida aparece una sección "TODO pendientes" con la lista ordenada. Si prefieres que cuenten como fallos, añade `--todo-fail`. Un TODO implementado pero incorrecto sí sale como **FAILED**, con el mensaje del `assert`; y recuerda que un `FutureWarning` o `DeprecationWarning` emitido desde `src/` también hace fallar el test.

Con los capítulos 1 a 4 terminados, `python -m pytest tests/test_walkforward.py` debería dar al empezar **1 passed, 20 skipped**: el que pasa es `test_generate_windows_rejects_overlapping_tests`, porque la validación ya está hecha. Tras el 5.1, 7 passed; tras el 5.2, 10; tras el 5.3, 15; tras el 5.4, **21 passed**. Si algún test sale skipped con un TODO de un capítulo anterior (por ejemplo `TODO 3.3 · bollinger_mean_reversion` en el test de tipos de `grid_search`, o `TODO 4.3 · sharpe_ratio`), resuélvelo primero: el walk-forward depende de todo lo anterior. `-m extension` no ejecuta nada de este capítulo; lo tienes aquí para cuando llegues al 06.